IBM Bob Premium Package for i

Participant Guide: Foodservice Distribution Use Case

Duration	60 minutes. Workshop slot: Lab 2, 11:30 to 12:30, after Lab 1 (foundational Bob overview).
Audience	IBM i / AS/400 developers and team leads who maintain an in-house order, pricing or inventory system. No prior AI tooling experience needed.
Format	Facilitator-led. Participants drive their own Bob IDE.
Environment	IBM i 7.6 LPAR with the DIST400 sample application restored. One user profile and one library per participant: LAB01 owns DIST401, LAB02 owns DIST402, and so on.

Your participant card

Your facilitator gives you a two-digit ID. Write it here and use it everywhere this guide says nn.

Your ID (nn)	put your ID here
User profile	LABnn
Key file	LABnn.pem, given by your facilitator
Your library	DIST4nn

Nothing you do in DIST4nn can affect anyone else. Your profile owns the library, your current library is already set to it, and your home directory on the server is yours alone.

What participants will do

Exercise	What you will do	Use case it answers	Time
0	Connect Bob to IBM i and have it write an overview of the library	Application documentation generation	4 min
1	Understand a 1998 pricing program you have never seen	Code documentation generation	9 min
2	Find out what a customer-number expansion would break, then draw the application flow	Interface and dependency mapping; application flow documentation	10 min
3	Map the database with one command	Data model documentation	5 min
4	Extract the pricing rules nobody wrote down	Business process documentation	9 min
5	Modernize the pricing program and compile it	Modernization with a reviewable audit trail	11 min
6	Write the runbook for the nightly job and pull the audit evidence	Operational documentation and runbooks; audit evidence and compliance support	7 min
7	Wrap-up and hackathon starting points		2 min

The theme running through this lab: recommended starting points for documenting and modernizing an RPG application. The exercises run in the order a team should actually work: generate the documentation that does not exist, map the dependencies, extract the business rules, then modernize with evidence at every step. Exercises 1, 4 and 5 all work on the same program, OE005, so by the end the room has understood it, documented it, modernized it and proved it still prices the same.

How to read the boxes A box captioned "Paste into the Bob chat" is a prompt: copy it exactly into Bob's chat panel and press Enter. A box captioned "Type in the terminal" is a command for a terminal window, not for Bob. A box captioned "What you should see" is Bob's expected output; do not type it.

About the sample application

DIST400 is a fixed-format RPG order-to-cash system for a broadline foodservice distributor, written in the style of a 1998 shop. Customers (restaurants, schools, hospitals) order cases of items. Items live in dry, cooler or freezer zones. Some items are catch-weight: sold by the pound, with the real weight only known when the case is picked. Pricing comes from a customer contract if one is in effect, otherwise from the customer's price class, otherwise list. Orders are cut off nightly and printed as route sheets.

Object	What it is
CUSTMAST, ITEMMAST, INVBAL	Customer master, item master, on-hand by warehouse
CONTRACT, PRICELST, SUBITEM	Contract prices with effective dates, price-class list prices, substitution items
ORDHDR, ORDDTL	Order header and lines
OE001, OE005, OE010, OE020	Order entry, price lookup, allocation and substitution, pick confirm
CM001, IN001, RP010	Customer maintenance, inventory inquiry, nightly route sheet

Key concept: modes

Spend three minutes here before Exercise 0. Modes are the concept everything else builds on.

A mode is a persona, a toolset and a set of permissions. Same Bob, different expert.

Mode	Persona	Use it for
IBM i Developer	Senior IBM i developer	RPG, DDS, CL, modernization, compiling, workflows
IBM i Database	Db2 for i expert	SQL, schema, indexes, performance, /erd
Your own custom modes	Whatever the team needs	Onboarding, code review, shop standards

Why IBM's modes matter Anyone can prompt an AI to act like an RPG expert. What you cannot write into a prompt is the ability to read a source member out of QSYS, compile it, or query Db2. IBM's modes ship roughly fifty encoded IBM i skills plus real tools, grounded in IBM's own documentation. A custom mode changes how Bob talks. IBM's modes change what Bob can do.

Exercise 0: Connect to IBM i and have Bob write the library overview

Goal: establish the connection everything else depends on, then have Bob document the application before anyone reads a line of code.

> **Start with an empty workspace.** In Bob choose File, then Open Folder, and pick a new, empty folder, for example one named lab-LABnn. Never open the lab kit folder or any folder that holds source. If Bob can see a local copy of the code it reads that instead of the IBM i, and the exercises stop showing what they are meant to show.

1. In Bob, click the IBM i icon (plug) in the left Activity Bar.
2. Click New Connection.
3. Enter the host, port 22 and your user profile LABnn. Leave the password empty and point the Private Key field at the LABnn.pem file your facilitator gave you. The lab IBM i accepts keys only; a password works only if your facilitator tells you they turned password login on.
4. Click Connect. On a first connection Bob asks whether to create /home/LABnn/.vscode with permissions 700: choose Yes. Then wait for the green confirmation.
5. In Object Browser click the filter icon (the funnel) to create a filter. The name is only a label; the Libraries field is what matters. Enter: Filter name DIST4nn, Filtering type Simple, Libraries DIST4nn, Objects *, Object types *ALL, Members *, Member type *, Member text *, and leave the two date fields blank. Save. The tree now shows DIST4nn with its programs, data files and six source files. If you see QGPL or only source files, edit the filter: Libraries or Object types is still at its default.
6. Right-click DIST4nn. It is already your current library because it is set on your profile; if the menu offers Set as Current Library, choose it.
7. Open the Bob chat panel, set the mode to **IBM i Developer**, and paste the prompt below, substituting your library name. Bob's own /init command only documents the folder open on your computer, which is empty on purpose, so the prompt asks for the same thing from the server.

Paste into the Bob chat

```
Scan library DIST4nn on the IBM i. List every program, file and source member, read
the source, and write a file named AGENTS.md in this workspace that describes what the
application is, which programs call which, which files each program reads or writes,
and the conventions you notice. Everything is on the server; do not ask me to add
files.
```

What it does

Bob lists the objects in your library, reads the members off the server and writes AGENTS.md into your empty workspace: what the application is, which programs call which, which files each one uses, and the conventions it found. It keeps working while you start Exercise 1. Come back and read it at the end of Exercise 1; it is the application overview nobody ever wrote. If Bob answers in the chat instead of writing the file, that is fine; the content is the same.

> ✓ **CHECKPOINT** You can expand DIST4nn and see programs (*PGM), files (*FILE) and source files QRPGSRC, QRPGLESRC, QDDSSRCF, QDDSSRCD and QCLSRC, and the overview is being written.

Exercise 1: Understand a program you have never seen

Goal: get a useful explanation of unfamiliar 1998 RPG in under a minute.

> *The scenario: a customer called to dispute an invoice price. The analyst asks you how pricing works. The program that prices every line is OE005, and nobody remembers writing it.*

1. In Object Browser, under your DIST4nn filter, scroll past the programs and data files to the six source folders near the bottom. They are shown in lowercase: qclsrc, qddssrc, qddssrcd, qddssrcf, qrpglesrc, qrpgsrc. Expand qrpgsrc, labelled QRPGSRC source.
2. Click oe005.rpg, labelled Price lookup, to open it. Read it for fifteen seconds. Do not worry about understanding it. The source must come from the server this way; the src folder in the lab kit on your computer holds the same code, but Bob needs to read it from the IBM i for these exercises to work.
3. Open the Bob chat panel and set the mode to **IBM i Developer**.
4. Type the prompt below.

Paste into the Bob chat

```
What does this program do?
```

What to expect

Bob should name the program's purpose, its six parameters and the four files it reads, and quote at least one detail that appears nowhere in the program itself, something it could only know by reading a file definition.

Watch Bob's status lines while it works

> *"Let me read the active file first."*
>
> *"Now let me look at the CONTRACT and ITEMMAST file definitions to understand the fields."*

This is the moment worth pausing on. Bob read the program, saw that the fields it uses are defined externally in DDS, fetched those file definitions too, and combined them. That is how it can tell you CEXPDT is a six-digit date, something that appears nowhere in the program. Pasting the program into a general-purpose chatbot would have missed it, because you would have had to know the DDS existed.

Now go deeper

Paste into the Bob chat

```
How does the program decide whether a contract price is still in effect?
```

Expected: a specific description of how two six-digit dates are compared, and the year in which the comparison stops working. If Bob does not name the year, the next prompt makes it.

Then ask about 2050

Paste into the Bob chat

```
What happens to a contract that expires in 2050?
```

Expected: Bob calls it a bug, names the year it starts, and may notice that some contracts in the file are already on the wrong side of it.

Before moving on, open the AGENTS.md from Exercise 0 in your workspace, or scroll back to that answer if Bob gave it in the chat, and find OE005 in it. The one-line description there should match what Bob just told you.

✓ **CHECKPOINT** You can explain to the analyst where a line price comes from, without knowing RPG, and you have a written application overview you did not write.

Exercise 2: What breaks if I change this?

Goal: see the full blast radius of a change before making it, then get the application flow drawn from the code.

> *The scenario: an acquisition means customer numbers will no longer fit in six digits. Sounds like a one-field change to CUSTMAST.*

1. Stay in **IBM i Developer** mode.
2. There is nothing to set for this one. The prompt names your library and Bob scans it on its own, as it did in Exercise 0; the Object Browser has no library-level context option, and none is needed.
3. Paste the prompt below, substituting your library name.

Paste into the Bob chat

```
If we expanded the customer number in DIST4nn from 6 to 8 digits for the acquisition,
what would break? Show me every program, file and screen that carries it, and draw a
diagram.
```

What a good answer looks like

The same value under at least four different field names, across database files, logical files, programs, screens and the printer file, with a diagram, a total count and a migration order. The total will almost certainly be short by three objects; the follow-up below finds them.

Only if Bob found fewer than four names for the field

Skip this if it found the renamed fields on its own. Otherwise paste:

Paste into the Bob chat

```
The logical file calls it CUSNBR, the order and contract files call it OCUST and
CCUST, and the screens call it SCUST. Find those too and give me the new total.
```

Now ask the question that catches the missing three

Bob has almost certainly listed OE010, OE011 and the logical file ORDHDRL1 as unaffected, because they never use the customer number. Paste this whether it did or not:

Paste into the Bob chat

```
What happens to OE010 and OE011 the first night the cutoff runs after ORDHDR is
rebuilt with the new format? They never use the customer number.
```

Expected: Bob explains that the record layout of a file is frozen into each program when it is compiled, so both programs must be recompiled even though their source does not change by a character, and it adds them to the list. Now read its description of what goes wrong. If it says nothing crashes and the data quietly goes wrong, that is only true for a file whose level checking was switched off. Yours has it on, as every file does by default, so the nightly job would fail at open with a level check instead. Bob had the fix right and the symptom wrong; the platform knowledge was still yours to supply. Make it check:

Paste into the Bob chat

```
Does ORDHDR in DIST4nn have level checking on? Check the file attribute, then tell me
again what actually happens the first night, and whether that changes the fix.
```

Expected: Bob checks the file, reports LVLCHK(*YES), corrects itself to a level-check failure at open, and keeps the same fix: recompile the two programs and rebuild the logical file.

The point of this exercise Search this code for the field name you know and you get one file. The real answer is much larger, because the value is called something different at every layer, and some programs are affected without ever using it. Miss one of those and a batch job fails in the middle of the night, or, if level checking was switched off, quietly reads the wrong bytes.

Now draw the flow

Paste into the Bob chat

```
Draw the order-to-cash flow of DIST4nn as a diagram: from order entry through pricing,
allocation, pick confirm and the nightly cutoff. Show which program calls which, and
which files each one reads or writes.
```

Expected: a diagram that shows order entry calling the pricing program, the nightly cutoff job driving allocation and the route sheet, and the files between them. Every arrow should come from a CALL or a file declaration in the source; there is no design document.

✓ **CHECKPOINT** You can name at least six programs affected by a change to one database field, including two that never use it, and you have a flow diagram of the application.

Exercise 3: Map the database

Goal: see the second built-in mode, and generate the data model documentation that does not exist. Five minutes: run the command, look at the diagram, move on.

1. Switch the chat mode to **IBM i Database**. Note that this is a different persona, not a different tool.
2. Type the command below, substituting your library name.

Paste into the Bob chat

```
/erd DIST4nn
```

What it does

Queries Db2's own system catalogue views, SYSTABLES, SYSCOLUMNS, SYSCST and SYSREFCST, then draws the relationships, marking primary keys, foreign keys, unique constraints and cardinality. It is reading the live database, not guessing.

Expect eight tables with the order lines in the middle, and look at whether any foreign keys are declared at all.

✓ **CHECKPOINT** You have a diagram of a database nobody documented, generated from the live system.

Exercise 4: Extract the pricing rules nobody wrote down

Goal: turn the program from Exercise 1 into a rules document a pricing analyst can read. This is the exercise participants remember.

> *The scenario: the analyst from Exercise 1 is back. They do not want an explanation of the code. They want the rules, in order, in English, so they can check them against the contract with the customer.*

1. Switch back to **IBM i Developer** mode.
2. In Object Browser, under DIST4nn, expand qrpgsrc and click oe005.rpg, then click its editor tab so it is focused.
3. Click the Run Workflow icon, the play triangle at the top of the chat panel. It first asks where to run the workflow: choose **Run Workflow in Library List**. The Bob workflows panel opens with thirteen workflows.
4. Scroll to the bottom and click **Start** on **Business Rules Extraction**. Not the similar one named Extract business rules higher up; that is Bob's generic workflow, and the one at the bottom is the IBM i version that reads the member off the server.
5. Click **Get Started**. Bob switches itself to IBM i Developer mode, reports "QSYS workspace detected", and asks whether to start the workflow for DIST4nn/QRPGSRC/OE005.rpg, the member in your active tab. Leave Yes selected and click **Continue**.
6. The workflow has eight steps and runs on its own for about three minutes: it parses the program, lists the four files it depends on, CUSTMAST, ITEMMAST, CONTRACT and PRICELST, extracts the rules from the main line and from the WINDOW subroutine, writes the report and draws three diagrams.
7. At the end it asks where to save the report, with an IFS path already filled in: /home/LABnn/dist4nn/docs/business-rules/OE005-business-rules.md. Accept it and click **Continue**, then click **Open File** when it reports the file saved. The file is on the IBM i, in your home folder, so you can also open it from the IFS Browser in the IBM i panel later.

What to expect

Bob writes an eight-section report and opens it in the editor when you click Open File. The rules are in section 2, one named subsection each, with a summary table at the end of the section. Expect six, in the order the program applies them. Count them, then look for the one that is in no spec and could not be guessed from the screens. There is at least one.

✓ **CHECKPOINT** You have a rules document, generated from the source, that names a rule no one in the room knew existed.

Exercise 5: Modernize and compile

Goal: convert the 1998 fixed-format pricing program to modern free-format and build it on the server, then prove it prices the same.

> **Before you start: in Object Browser, under DIST4nn, expand qrpgsrc, click oe005.rpg, and click that editor tab so it is focused.** The workflow runs against the active editor: an IBM i member is read off the server; anything else makes it ask for a file path on your laptop.

1. Stay in **IBM i Developer** mode.
2. With OE005 open and focused, type the prompt below.
3. Bob answers with an RPG Modernization card: click **Start workflow**. The welcome shows the pipeline it will follow, from selecting the member and detecting its type to converting to free format in a compile loop and a summary report. Click **Get Started**.
4. It detects the member in your active tab and asks whether to start the workflow for DIST4nn/QRPGSRC/OE005.rpg. Leave Yes selected and click **Continue**. It recognises OPM RPG, writes a temporary ILE fixed-format copy named OE005_T into QRPGLESRC, and starts a subagent to compile and fix that copy first.
5. Bob asks before it runs anything on the server, and the compile loop runs many commands, so answer the first prompts in the way that covers the rest. When "Tools awaiting approval" shows a CRTBNDRPG command, click **Approve for task**. When it says an SQL statement with CALL QSYS2.QCMDEXC requires explicit approval, choose **Yes, and remove "CALL" from denied SQL statement types**. Answer "once" or "just this time" and it stops at every compile.
6. When it asks you to confirm the output member location, the suggestion is DIST4nn/QRPGLESRC/OE005.RPGLE: keep **Use suggested location** and click **Continue**.
7. The free-format conversion then runs as a nine-step task: it loads its RPG skills, analyses the member, writes the conversion and compiles until clean. Expect it to rewrite the source at least once. The parameters of this program are one-field data structures, which collide with the parameter names in the new procedure interface, and Bob works that out on its own; the thinking you see is it doing so. Let it finish.

Paste into the Bob chat

```
Can you modernize this program?
```

You are in the right place if the workflow says "Active RPG Member Detected" with your library's member and never asks for a path on your computer.

What to expect

Bob writes free-format ILE RPG to QRPGLESRC, then compiles it on the server:

What you should see

```
CRTBNDRPG PGM(DIST4nn/OE005) SRCFILE(DIST4nn/QRPGLESRC) SRCMBR(OE005)
```

Bob's summary report ends with the compile status, severity 00, and the number of attempts it needed; five is normal for this program, because its parameters are one-field data structures that clash with the names in the new procedure interface until Bob sorts that out.

At the bottom of the chat panel, click File Changed to see the before and after diff. The Y2K window from Exercise 1 is now a free-format subroutine you can read without a column ruler. Exercise 6 will show you the new object's attribute from the catalogue.

> **Why this exercise matters** Bob did not suggest code for you to copy somewhere. It wrote source onto the server and compiled it there; severity 00 means it built clean. The new program exists because you approved each step, and it is still RPG, still on IBM i, still called by OE001 with the same parameters.

Now prove it still prices the same

Paste into the Bob chat

```
Compare the original OE005 in QRPGSRC with the modernized version in QRPGLESRC.
Confirm the pricing precedence is identical: contract, then price class, then list,
with the catch-weight and class S adjustments. For customer 100230, item 201410,
quantity 5, order date 15 October 2026, what unit price and price source does each
version return?
```

Bob should report that both versions return the same unit price and the same price source for that customer and item.

Only if Bob reports a difference

That is a real finding, not a lab error. Paste:

Paste into the Bob chat

```
Show me the line in each version that produces the different result, side by side, and
tell me which one is wrong.
```

✓ **CHECKPOINT** A program written in 1998 has been converted, compiled and verified against a known price, with a reviewable diff.

Exercise 6: Runbook and audit evidence

Goal: produce the two documents operations and audit keep asking for, from the system itself.

> *The scenario: the nightly cutoff job failed at 2 am last month and the only person who knew how to restart it was on vacation. Operations wants a runbook. Separately, internal audit wants to know what changed in this library this quarter and who changed it.*

Part A: the runbook

1. Stay in **IBM i Developer** mode.
2. In Object Browser, under DIST4nn, expand the folder qclsrc, labelled QCLSRC source, and click ordcutof.clp, labelled Nightly order cutoff, to open it.
3. Type the prompt below. This is the longest single answer Bob writes in the lab, about two minutes, with a flowchart and SQL checks at the end. If a red Request Failed box appears instead, click **Retry**: the failure is on Bob's service side, not in your prompt, and a retry or two gets through.

Paste into the Bob chat

```
Write a runbook for this nightly job for the operations team: what it does step by
step, what must be true before it runs, what each step's failure looks like on the job
log, and how to restart safely from each step without allocating any order twice.
```

What to expect

Expect a four-step runbook, guard, allocate, print, close out, that says for each step whether it is safe to run again and why, and cites the line in the source that makes the rerun safe. A good answer goes one better. The header comment in OE010 claims a rerun is always safe, but the program updates the inventory balance before it saves the line's shipped quantity, so a job that dies between those two writes leaves one line that a rerun would allocate twice. If Bob reports that exception and tells operations how to check for it, it is right, and it found something the comment got wrong.

Part B: the audit evidence

1. Switch the chat mode to **IBM i Database**.
2. Type the prompt below, substituting your library name.

Paste into the Bob chat

```
Which objects in DIST4nn were created or changed today? For each one show the user who created it, the owner, when it was created, the source file and member it was built from, and the program attribute.
```

What to expect

Two rows. The first is OE005: type *PGM, attribute RPGLE, created today by LABnn, your own profile, built from DIST4nn/QRPGLESRC member OE005. The second is the source file QRPGLESRC itself, still created by the facilitator's profile on the build date but changed today, because the workflow added and removed members in it. Bob may add that the temporary member and program OE005_T were created and deleted during the workflow, so they no longer appear, and that for an ILE program it took the source location from the bound module rather than the object description. Every other object in the library was created by the facilitator's profile on the build date from QRPGSRC. Bob is reading the object statistics from the QSYS2 catalogue services, the same data an auditor would ask you to export.

Why this exercise matters Together with the diff and the approval trail from Exercise 5, that row is the audit evidence: who, what, when, from which source, and what changed. Bob gathers the evidence and writes the documents. It is not a compliance tool and the guide does not claim it is; the controls are still yours.

✓ **CHECKPOINT** You have a runbook for a job you did not write, and an audit row for a change you did make.

Exercise 7: Wrap-up and hackathon starting points

Six things participants just did, none of which base Bob can do:

1. Had Bob write an overview of a whole library from the server, and read source directly off IBM i with no copy-paste.
2. Traced a customer-number change across every object that carries it, under every name it uses, and drew the application flow from the code.
3. Generated a database diagram from the live system catalogue.
4. Extracted the pricing rules from a 1998 program, including one nobody knew about.
5. Modernized and compiled that program on the server, with approval at each step, and proved it prices the same.
6. Wrote a runbook for the nightly job and pulled the audit evidence for the change, from the system itself.

Starting points for the hackathon

Each of these runs in the same DIST4nn library and picks up where an exercise left off.

Topic	Starts from	What you would do
Library-wide dependency map	Exercise 2	Open the Understand Dashboard, create an application for DIST4nn, run the scan, and compare its dependency graph to what Bob found by hand.
Prove it with tests	Exercise 5	Run the RPGUnit Test Plan Creation workflow on OE005, then Test Suite Implementation. The six rules from Exercise 4 are the test cases.
Your own reviewer	Exercise 5	Build a custom mode in the ibm-i-developer group called Shop Standards Reviewer with your team's rules, and have it review the modernized OE005.
Make the cutoff faster	Exercise 6	Give the order-selection query from ORDCUTOF to /review, then run the Index Advisor workflow and apply its recommendation.
Operational intelligence	Exercise 6	Have Bob write a SQL service query for jobs, locks or spool backlog, then schedule it with the job scheduler and route the output to a data queue or email.
Green screen to web	Exercise 1	Generate a React page for inventory inquiry against IN001's files, running on the LPAR in PASE.

Troubleshooting

Symptom	Cause and fix
Bob's answer starts with "Read file src/..."	Bob is reading a local copy from its workspace. Open an empty folder as the workspace, reconnect, ask again.
IFS Browser shows /home/ITZUSER with a warning	The connection's home directory was copied from another connection. In the connection settings set Home Directory to /home/LABnn; the workflows write there.
Bob says "paste the source code here"	Not connected. Reconnect via the IBM i icon. Most common issue in this lab.
"IBM i MCP tools are currently disabled"	Same cause: no active IBM i connection.
Object Browser shows QGPL, or looks empty	The filter's Library field is still QGPL, or its object type is still *SRCPF. Edit the filter: Library DIST4nn, object type *ALL.
Workflow asks for a local file path	The active editor is not an IBM i member. Cancel, click the OE005 tab, re-run.
Business Rules Extraction is not in the Bob workflows panel	Premium Package not fully installed. Fall back to the prompt "List every business rule in this program, numbered, in order" in IBM i Developer mode.
/erd returns nothing	Reconnect the IBM i connection and retry.
Your library shows changes you did not make	You connected with someone else's profile. Profile and library numbers must match your ID. Reconnect as LABnn.
The Exercise 0 prompt says the workspace is empty and asks for files	Bob read the prompt as being about the local folder. Make sure the mode is IBM i Developer and the prompt names DIST4nn on the IBM i; if it persists, ask it to list the objects in DIST4nn first.
A red Request Failed box with AI_APICallError	Bob's model service failed, not your prompt or your connection. Click Retry; it can take two or three tries. Anything Bob already wrote is kept: click Show all next to files changed.
Audit query in Exercise 6 returns no rows	Exercise 5 did not recompile, or you are in a different library, or the IBM i clock runs on UTC and has already passed midnight. Ask for "in the last 24 hours" instead; ask for "since the library was built" to see the facilitator's rows.
Compile fails in Exercise 5	Do not debug live. Show the diff instead; the generated code is still the point.
Modes selector missing IBM i modes	Premium Package not enabled. The lab cannot run without it.

Exercises 2, 3 and 6A are independent. Exercises 4 and 5 build on 1; if 1 fails, open OE005 and move on. Exercise 6B needs the recompile from Exercise 5.